RICHTLINIE

Projektstruktur

Verzeichnisse, Skripte und erzeugte Dateien des Gesamtprojekts: welcher Pfad wofür zuständig ist

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205-Projektstruktur.md`	0.03-014
Stand	19.10.2026	
Umfang	13 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_projektstruktur_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

Inhaltsverzeichnis

1. Oberste Ebene 3
2. Konventionen für Tests, Protokolle und virtuelle Geräte 11
3. Teilprojekt Assembler 12
3a. Teilprojekt BIS 13
4. Offene Punkte 13
5. Nachführung Stufe 4 (19.10.2026) 13
6. GRAFIK-Verzeichnisse (Stufe 6) 13

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Version: 0.03-014

Dieses Dokument erläutert die Verzeichnisstruktur des Gesamtprojekts C 8205: welches Verzeichnis welchen Zweck hat und in welchem Dokument die Einzelheiten stehen. Es wiederholt keine Details aus den Teildokumenten, sondern verweist nur auf sie (wie "C8205-Interpreter-Überblick_und_Gesamtaufbau.md"). Erzeugte Dateien werden nur nach Endung beschrieben, nicht einzeln aufgezählt.

Das Projekt hat drei Teilprojekte: den **Interpreter** (Python, Wurzel des Projektverzeichnisses), den **Assembler** (C, Unterverzeichnis `Assembler/`) und das **Bausteininterpretationssystem BIS** (Programmsystem für den C 8205; derzeit nur Dokumentation, s. Abschnitt 3a).

1. Oberste Ebene

Pfad	Zweck	Maßgebliches Dokument
`c8205_interp.py`	Python-Interpreter (Knotenautomat)	`C8205-Interpreter-Python_Interpreter.md`
`Hilfsprogramme/`	Ordner für die Hilfsprogramme des Interpreter-Teilprojekts: `c8205_lochstreifen_anzeige.py`, `c8205_lochstreifen_eingabe.py`, `c8205_lochstreifen_umwandlung.py` und `c8205_befehlssweep.py` (die vier folgenden Zeilen). Sie liegen nicht mehr in der Wurzel; `c8205_interp.py` und die Wrapper `c8205_interpreter.sh`/`.bat` bleiben dort	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
`Hilfsprogramme/c8205_lochstreifen_anzeige.py`	Hilfsprogramm (Interpreter-Teilprojekt): liest einen virtuellen Lochstreifen (Datei oder Verzeichnis, Format wie `--leser1`/`--leser2`) und schreibt ihn als Klartext in einem scrollbaren Fenster aus (Schalter `--LS-LESER`). `Um. KB`/`Um. GB` wirken nur, `Irrung Zeichen`/`Irrung Satz` löschen im Text. Eigene Programmversion (`--version`)	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
`Hilfsprogramme/c8205_lochstreifen_eingabe.py`	Hilfsprogramm (Interpreter-Teilprojekt), Gegenstück zur Anzeige: erzeugt einen virtuellen Lochstreifen aus Tastatureingaben in einem Fenster (Schalter `--LS-STANZER`); Ergebnis geeignet für `--leser1`/`--leser2`. Eigene Programmversion (`--version`)	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
`Hilfsprogramme/c8205_lochstreifen_umwandlung.py`	Hilfsprogramm (Interpreter-Teilprojekt): wandelt eine Textdatei in einen binär kodierten virtuellen R300-Lochstreifen um (`--IN-BIN`, Vorgabe) oder umgekehrt (`--IN-TXT`); Aufruf `python3 c8205_lochstreifen_umwandlung.py --STANZER AUSGABE --LESER EINGABE [--IN-BIN | --IN-TXT]`	Kopfkommentar der Datei
`Hilfsprogramme/c8205_befehlssweep.py`	Vollständigkeitstest über alle 512 Operationscodes. Standardmäßig sucht er `c8205_interp.py` im eigenen Ordner; liegt er in `Hilfsprogramme/`, ist `--interp ../c8205_interp.py` anzugeben	Kopfkommentar der Datei

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`c8205_grafik.py`, `c8205_grafik.sh`, `c8205_grafik.bat`	Treiber des Modus GRAFIK (Pult-Fenster, SW-Fenster, Töne); Schalter u. a. `--STUMM`, `--TON=pppp` (Pult, SW, LL, LS; Vorgabe 5555), `--leser1`, `--leser2`, `--stanzer1`, `--HWCHECK`; Rückkehrcode 16 = Start nicht möglich (z. B. kein Abspielprogramm)	`C8205-Interpreter-GRAFIK_Beschreibung.md` (Abschnitte 1, 2, 6)
`grafik-interpreter-module/grafik_*.py`	Module des Modus GRAFIK (liegen seit G5-002 in diesem Unterordner; `c8205_grafik.py` und die Tests tragen ihn in `sys.path` ein). Kern und Geräte: `grafik_trommel`, `grafik_geraete` (Leser, Stanzer), `grafik_k0`. Fenster: `grafik_pult`, `grafik_pult_fenster`, `grafik_sw`, `grafik_sw_fenster`, `grafik_hwcheck`, `grafik_fensterlage`, `grafik_tastatur`, `grafik_haltzeit`, `grafik_debuglog`. **Töne (Stufe 4):** `grafik_tonmisch` (Mischer, Python), `grafik_tonmisch_c` (Mischer über das C-Modul), `grafik_schreibwerk` (Schreibwerkzeit 80 ms ±10 ms, Zeichenklang 100 ms, TAB 400 ms, WRZL 360 ms), `grafik_geraeteton` (Leser, Stanzer), `grafik_rechnerton` (Hochlauf, Grundgeräusch, Auslauf); `grafik_ton` bleibt für Windows (Stufe 2)	`C8205-Interpreter-GRAFIK_Beschreibung.md` (Abschnitte 5, 6, 11)
`c-module/`	C-Module: `trommel`, `bahnsperre`, `unterbrechung` und `tonmisch` (Mischer in Blöcken zu 10 ms mit eigenem C-Thread, statische Bibliothek, fest im Erweiterungsmodul), Testrahmen `testframe_*.c`, `Makefile`; `c8205_api.h` (Exportmakro der gemeinsamen Bibliothek), `plattform.h` (dünne Schicht für Linux und Windows, nur von `trommel.c` eingebunden; der Windows-Zweig ist nicht übersetzt und nicht getestet) und `build_msvc.bat` (Bau von `trommel.dll` mit Visual Studio, nicht getestet); `python/` mit dem Erweiterungsmodul `c8205_trommel` (`setup.py`, `c8205_trommel_modul.c`)	`C8205-Interpreter-GRAFIK_Beschreibung.md` §7

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
ton/	Klänge und Testrahmen: sw_toene/ (24 Zeichenklänge, tab_400.wav, wrzl.wav, Leserklänge ls_*.wav; tab_kurz.wav/tab_lang.wav nur Quelle), toene_rechner_stanzer/ (Stanzerklänge ls_motor.wav und ls_dauerstanzen.wav; die dortigen Kopien von Netz ein/aus und Grundgeräusch liest kein Programm), die drei Klänge des Rechners netz_ein_fade_in.wav, netz_aus_fade_out.wav und rechner_grundgeraeusch.wav liegen im Ordner ton/ selbst, aufnahmen/ (MP3-Quellen), sw_ton.py (selbständiger Testrahmen: Schreibwerkzeit 80 ms ±10 ms, Zeichenklang 100 ms), SW_Ton_Beschreibung.md	ton/SW_Ton_Beschreibung.md
pult/	Bilder und Manifest des Bedienpult-Fensters: pult_manifest.json, bild/ (c8205_bedienpult.png, das Bild des Pults) und grafiken/ (Lampen, Dioden und Schlüsselstellungen als PNG)	C8205-Interpreter-GRAFIK_Beschreibung.md §3, §4
testscripte/	(Ergänzung zur obigen Zeile) Prüfungen des Modus GRAFIK: grafik_alle_tests.sh (alle Blöcke, --schnell ohne den Gleichwertigkeitstest), grafik_stufe4_tests.py (Töne), grafik_sw_ende_zu_ende.py, grafik_sw_abstand_tests.py, grafik_tastatur_tests.py u. a. Dazu das Messwerkzeug k0_schatten_haken.py (zu Interpreter 0.03-016, nur auf eine Kopie anzuwenden) und die Prüfung des Eingabeprogramms r300_eingabe: r300_eingabe_erzeugen.py (erzeugt Quelltext, Testauftrag, K0-Eingabedatei und die Klickspur Tests/r300_eingabe_klickspur.txt, die derzeit nicht im Projekt liegt), r300_eingabe_trockenlauf.py (ohne Tkinter), r300_eingabe_grafik.sh/.bat (GRAFIK-Lauf mit --HWCHECK und --WIEDERGABE), r300_eingabe_lauf_und_auswertung.sh/.bat (Lauf und Auswertung in einem Schritt) und r300_eingabe_log_auswerten.py (wertet das Debug-Log aus)	C8205-Interpreter-GRAFIK_Pruefverfahren.md §3
c8205_interpreter.sh, c8205_interpreter.bat	Wrapper: starten c8205_interp.py für eine Testdatei und schreiben das Protokoll nach Test-Protokolle/<Testname>.log	C8205-Interpreter-Python_Interpreter.md, Abschnitt 5.2

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`testscripte/`	Ordner für die Shell-Skripte, die Tests ausführen: `c8205_bis_test.sh`, `test_bis.sh`, `test2_bis.sh`, `test3_bis.sh`, `test.sh` und `compute_all_tests.sh` sowie das Python-Hilfsskript `erzeuge_leere_trommel.py` (die folgenden Zeilen). Jedes Skript wechselt zuerst in das Projektverzeichnis (eine Ebene über `testscripte/`) und arbeitet dort; es darf daher aus jedem Verzeichnis gestartet werden. Pfade von Testdateien (z. B. `Tests/pj2-bis_befehlsdruck.txt`) gelten relativ zum Projektverzeichnis. Die Wrapper `c8205_interpreter.sh`/`.bat` liegen weiterhin in der Wurzel	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f
`testscripte/erzeuge_leere_trommel.py`	Hilfsskript: erzeugt die Trommeldatei `virtuelle_Geraete/Trommel/trommel.txt` mit 4096 Zeilen, alle Zellen 0 (Schalter `--aus PFAD`). Setzt damit auch eine benutzte Trommel zurück; der Interpreter selbst setzt sie nie zurück	`C8205-Interpreter-Python_Interpreter.md`, Abschnitt 8.5
`testscripte/c8205_bis_test.sh`	Wrapper für Tests der BIS-Bausteine: führt zuerst `Tests/pj2-bis_minimalprogramm.test` aus (Minimalprogramm auf die Trommel), dann die angegebene Testdatei; Aufruf `testscripte/c8205_bis_test.sh TESTDATEI [Schalter ...]` (vom Projektverzeichnis aus)	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f
`testscripte/test_bis.sh`	führt die Grundtests des BIS nacheinander aus: setzt die Trommel mit `erzeuge_leere_trommel.py` zurück und startet dann das Minimalprogramm und die Tests zur Adressausgabe, Adresseneingabe, Befehlsdruck und `pj2-bis_druck_bz_ac` über `c8205_bis_test.sh`	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f
`testscripte/test2_bis.sh`	zweiter Satz BIS-Tests, gleicher Ablauf wie `test_bis.sh` (leere Trommel, Minimalprogramm, dann die Tests): weitere Fälle der Adresseneingabe (`buchstabe_wie_null`, `gb_in_ziffern`, `wr_anfang`), alle Tests `pj2-bis_kz_*` (Kennzeichen im Wort bzw. vor K1) und `pj2-bis_ps_*` (Programmsteuerung)	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f
`testscripte/test3_bis.sh`	erweiterte Fassung von `test2_bis.sh`: gleicher Ablauf (leere Trommel, Minimalprogramm, dann die Tests), aber mit mehr Fällen, u. a. zusätzlich `pj2-bis_adresseneingabe_bm_anfang`, `pj2-bis_adresseneingabe_zv_wie_null`, `pj2-bis_kz_bm_vor_k1`, `pj2-bis_kz_k1_1_punkt` und `pj2-bis_kz_k1_2_punkt`	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`testscripte/test.sh`	ältere Fassung von `test_bis.sh` (gleicher Ablauf, aber ohne den Test `pj2-bis_druck_bz_ac`)	–
`testscripte/compute_all_tests.sh`	führt den Interpreter für alle Testdefinitionen in `Tests/` aus; Soll-Dateien der Dateiprüfungen (Namen auf `-ausgabe.txt` oder `_erwartet.txt`) und die BIS-Tests `pj2-bis_*` (laufen mit `test_bis.sh` und `test2_bis.sh`) überspringt es	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 3b.1
`README.md`, `README.en.md`	Einstiegsseiten für GitHub (deutsch, englisch); keine eigenen Regeln, sie verweisen auf die Dokumente. Ausnahme von der Regel, dass .md-Dateien in `markdown-dateien/` liegen	`C8205-Interpreter-Projektueberblick.md`
`LICENSE`	MIT-Lizenz für das Gesamtprojekt (Originalunterlagen des C 8205 ausgenommen)	–

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`markdown-dateien/`	Unterordner mit den .md-Dateien des Projekts (Projektbeschreibung und Regelwerk, Quelle für Interpreter und PDFs): alle `C8205-Interpreter-*.md`-Dateien außer `C8205-Interpreter-BIS.md` und `C8205-Interpreter-Objektdatei_Listing.md` (beide im BIS-Ordner; darunter `C8205-Interpreter-PDF.md`, die Gestaltungsrichtlinie für alle PDF-Ausgaben, und `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`, die Dokumentation der beiden Lochstreifen-Hilfsprogramme), `baz-wirkungstabelle.md`, das Orientierungsdokument für neue Leser `C8205-Einstieg_fuer_Neueinsteiger.md` (gilt für das Gesamtprojekt, nicht nur für den Interpreter) sowie die beiden projektweiten Wurzeldokumente `C8205-Projektstruktur.md` (dieses Dokument) und `C8205-Versionierung.md` (Versionsnummern der Dateien). Die Rolle jeder Datei steht in der Dokumentenlandkarte von `C8205-Interpreter-Projektueberblick.md` (Abschnitt 6). Nicht hier liegen die Assemblerdokumentation (`Assembler/Dokumentation/`, s. Abschnitt 3) und die BIS-Dokumentation einschließlich des Objektdatei-Listings (`Bausteininterpretiersystem BIS/`, s. Abschnitt 3a). Jede .md-Datei existiert nur einmal. Im Unterordner `abbildungen/` liegen die Abbildungen der Dokumente (PNG, die selbst gezeichneten auch als SVG; eingebunden mit `![Bildunterschrift](abbildungen/Datei.png)`; Zeichenskript `zeichne_abbildungen.py` in `PDF/pdf-layout-scripte/`). Der Interpreter liest den Ordner über den Schalter `--mdverz` (Vorgabe: `markdown-dateien/` neben `c8205_interp.py`).	Wurzel: `C8205-Interpreter-Überblick_und_Gesamtaufbau.md`; PDF-Richtlinie: `C8205-Interpreter-PDF.md` (selbst); Versionsnummern: `C8205-Versionierung.md`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`PDF/`	Erzeugte PDFs (A4), je Dokument eine Datei `C8205-<Inhalt>_A4.pdf`; hier nicht einzeln aufgezählt. Das Assembler-PDF liegt dagegen in `Assembler/Dokumentation/` (s. Abschnitt 3), das BIS-PDF in `Bausteininterpretiersystem BIS/markdown-dateien/` (s. Abschnitt 3a) und das PDF der Testübersicht in `CSV/`. Die Skripte zum Erzeugen liegen im Unterordner `PDF/pdf-layout-scripte/`: je PDF ein Skript `erzeuge_<Inhalt>_pdf.py` (z. B. `erzeuge_knoten_pdfs.py` für `C8205-Knotengraph_A4.pdf` und `C8205-Knotentabellen_A4.pdf` aus den beiden Knoten-.md-Dateien, `erzeuge_rechnerbeschreibung_pdf.py` mit dem redaktionellen Text im Skript, `erzeuge_projektstruktur_pdf.py` für `C8205-Projektstruktur_A4.pdf`), `build_testhandbuch.py` für das Testhandbuch, die gemeinsamen Bausteine `pdflib.py`, der Markdown-Konverter `md2pdf.py` und das Zeichenskript `zeichne_abbildungen.py` für die selbst gezeichneten Abbildungen (Lampenfeld, Tastaturen; Ausgabe nach `markdown-dateien/abbildungen/`). Ein Skript ist jeweils beim Ordner seiner Quelle abgelegt: `Assembler/Dokumentation/pdf-layout-scripte/`, `Bausteininterpretiersystem BIS/markdown-dateien/pdf-layout-scripte/` und `CSV/pdf-layout-scripte/`; sie verwenden `md2pdf.py` und `pdflib.py` aus `PDF/pdf-layout-scripte/`.	Liste, Quellen und Namensschema: `C8205-Interpreter-PDF.md`, Abschnitt 8; Skript (Aufruf, Voraussetzungen): Abschnitt 7
`make_pdf.sh`, `make_all_pdf.sh`	Aufrufhilfen für die PDF-Erzeugung (in der Wurzel): `make_pdf.sh SKRIPT [Schalter ...]` sucht das Skript `SKRIPT` in einem der Ordner `pdf-layout-scripte` (`PDF/`, `Assembler/Dokumentation/`, `Bausteininterpretiersystem BIS/markdown-dateien/`, `CSV/`) und startet es mit den übrigen Schaltern (z. B. `--stand 12.10.2026`); `make_all_pdf.sh` erzeugt über `make_pdf.sh` nacheinander die PDFs einer festen Skriptliste	Kopfkommentar von `make_pdf.sh`

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`Bausteininterpretiersystem BIS/`	Teilprojekt BIS: Unterordner `markdown-dateien/` mit `C8205-Interpreter-BIS.md` (Dokumentation des BIS), `C8205-Interpreter-Objektdatei_Listing.md` (Objektdatei-Ausdrucke der Beispielprogramme, Vorgabebilder) und `C8205-BIS_A4.pdf` (PDF dazu) und dem Unterordner `pdf-layout-scripte/` mit `erzeuge_bis_pdf.py` (erzeugt das PDF)	Abschnitt 3a
`Tests/`	Testaufträge für den Interpreter (meist `.txt`; die Endung ist frei, der Assembler erzeugt `.test`)	`C8205-Interpreter-Durchführung_eines_Tests.md`
`Test-Ergebnisvergleiche/`	Soll-Dateien der Dateiprüfungen (Namen auf `-ausgabe.txt` oder `_erwartet.txt`), mit denen Tests ihre Ausgabe vergleichen; sie liegen nicht in `Tests/`	`C8205-Interpreter-Durchführung_eines_Tests.md`
`Test-Protokolle/`	Protokolle (`.log`) der Interpreterläufe	`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 8
`virtuelle_Geraete/`	Ablage der virtuellen Geräte für den Interpreter, je Gerät ein Unterordner: `Leser1/`, `Leser2/` (Lochstreifen für Leser 1 und 2), `Stanzer1/` (Ausgabe des Stanzers 1), `SW-Ausgabe/` (Schreibwerk-Ausgabe), `T-SW-Eingabe/` (Eingabe über Tastatur/Schreibwerk, Kanal K0), `Trommel/` (die Trommeldatei `trommel.txt`, der persistente Speicher des Interpreters); siehe Abschnitt 2	Mechanismus: `C8205-Interpreter-Durchführung_eines_Tests.md`, 6.2 und 6.3.2; Schalter: `C8205-Interpreter-Python_Interpreter.md`, 8.2 und 8.3; Trommel: dort 8.5 und `C8205-Interpreter-Durchführung_eines_Tests.md`, 2c

C 8205 – Projektstruktur

Quelle: C8205-Projektstruktur.md · im Wortlaut der Quelle

Pfad	Zweck	Maßgebliches Dokument
`CSV/`	Übersichtstabelle aller Tests: `test_tabelle.py` liest `Tests/` und `Assembler/` und schreibt `test_tabelle.csv`, `.md` und `.xlsx` (Schalter `--format` auch `html`) mit den Spalten Testdatei, Inhalt des Tests (aus den `#DOK`-Zeilen), Dateien der virtuellen Geräte bzw. Vergleichsdatei und zugehöriger ASM-Quelltext. Das Skript findet die Basis (den Ordner mit `Tests/` und `Assembler/`) selbst, auch wenn es aus `CSV/` gestartet wird; die Ausgabedateien liegen im Ordner des Skripts. `test_tabelle_csv_anzeigen.sh` öffnet `test_tabelle.csv` in LibreOffice Calc und markiert in der letzten Spalte die Zellen grün, deren Wert größer ist als der der Spalte davor. Außerdem liegen hier das PDF `C8205-Testuebersicht_A4.pdf` (die Übersicht als PDF) und im Unterordner `pdf-layout-scripte/` das Skript `erzeuge_testuebersicht_pdf.py`, das es erzeugt	Kopfkommentar von `test_tabelle.py`; Aufruf: `python3 test_tabelle.py [--basis PFAD] [--format csv md xlsx html]`
`Assembler/`	Teilprojekt Assembler	Abschnitt 3; Doku: `Assembler/Dokumentation/C8205_Assembler_Dokumentation.odt` (maßgeblich), `.md` (durchsuchbare Ableitung) und `C8205-Assemblerdokumentation_A4.pdf` (aus der `.md` erzeugt), gleicher Pfad

2. Konventionen für Tests, Protokolle und virtuelle Geräte

- **Testname:** `NNNN-<Operationscodes>.txt` in `Tests/` (z. B. `0004-210_544_020.txt`). Testaufträge aus Assemblerläufen heißen `asm_NNNN_<Programmname>.txt` (z. B. `asm_0001_bsp_5_16.txt`). Weitere Namensfamilien in `Tests/`: `bsp_…`, `sp-…`, `tk-…`, `st-…`, `int-…`, `ziffern_…`, `lb-…`, `ac-…`, `bz-…`, `acbz-…`, `wr-…`, `trommel-…` und `s1-…` bis `s4-…`; ihr Inhalt steht jeweils in den `#DOK`-Zeilen der Testdatei (Übersicht: `CSV/`). Das Protokoll trägt denselben Namen mit der Endung `.log` in `Test-Protokolle/`. Der Wrapper findet die Testdatei über den Namen.
- **Virtuelle Geräte:** Der Interpreter kennt die Geräte nur über die Schalter `--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe` und `--k0`. Diese nehmen einen Pfad (Datei oder Ordner) entgegen. Die Verzeichnisse `virtuelle_Geraete/Leser1/`, `…/Leser2/`, `…/Stanzer1/`, `…/SW-Ausgabe/` und `…/T-SW-Eingabe/` sind die dafür vorgesehene Ablage; **die Wrapper übergeben sie nicht**; die Testdatei darf die Zuordnung aber selbst als Zeile enthalten (`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2b). Sie wirken nur, wenn der Schalter beim Aufruf angegeben wird, z. B. `--leser1 virtuelle_Geraete/Leser1` oder `--k0 datei=virtuelle_Geraete/T-SW-Eingabe`.
- **BIS-Tests:** Tests, die einen BIS-Baustein prüfen, tragen das Präfix `pj2-bis_` (z. B. `pj2-bis_befehlsdruck.txt`, Soll-Datei `pj2-bis_befehlsdruck_erwartet.txt`). Sie setzen das Minimalprogramm auf der Trommel voraus, das `pj2-bis_minimalprogramm.test` zuerst laden muss (`c8205_bis_test.sh`); Einzelheiten: `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2f.
- **Trommel:** Die Trommel ist die Datei `virtuelle_Geraete/Trommel/trommel.txt` (4096 Zeilen). Anders als die übrigen Geräte ist sie nicht optional: Der Interpreter braucht sie bei jedem Aufruf. Vorgabe ist der Pfad neben `c8205_interp.py`; ein anderer Pfad wird mit `--trommel` angegeben (die Wrapper übergeben den Schalter nicht, es gilt die Vorgabe). Die Datei wird mit `testscripte/erzeuge_leere_trommel.py` erzeugt. Sie **behält ihren Inhalt** über Tests und Aufrufe hinweg: Die Zeilen einer Testdatei überlagern sie, nicht genannte Zellen behalten ihren Wert, jede Zelländerung eines Befehls wird sofort in der Datei nachgeführt. Der Interpreter setzt sie nie zurück. Tests, die Zellen prüfen oder lesen, die sie nicht selbst belegen, brauchen daher eine leere Trommel (`C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2c).

3. Teilprojekt Assembler

Der Assembler `c82ass` übersetzt Assembler-Quelltext (`.asm`) in Objektcode für das "Kleine Eingabeprogramm". Aufruf, Sprache, Makros und Beispiele stehen in der Assemblerdokumentation; hier nur die Struktur.

Pfad (unter `Assembler/`)	Zweck
`make_c82ass.sh`	Skript zum Erzeugen des Assemblers unter Linux
`make_c82ass.bat`	Windows-Pendant zu `make_c82ass.sh` (erzeugt `_Windows/c82ass.exe`; kein Wrapper zum Ausführen)
`c82ass.sh`	Ausführen des Assemblers
`asm_alle.sh`	assembliert alle `.asm`-Dateien unterhalb eines Verzeichnisses (Vorgabe: `Assembler/`) mit `c82ass.sh`; meldet nur Fehler
`asm_aufraeumen.sh`	löscht in den Projektordnern die vom Interpreter nicht benutzten Dateien `.log`, `.mqo`, `.obj`, `.sym` (Trockenlauf ohne Schalter, `--loeschen` löscht)
`Dokumentation/`	Assemblerdokumentation `C8205_Assembler_Dokumentation`: `.odt` (maßgeblich), `.md` (durchsuchbare Ableitung daraus, kein eigenes Regelwerk), `C8205-Assemblerdokumentation_A4.pdf` (aus der `.md` erzeugt, nach der PDF-Richtlinie); im Unterordner `pdf-layout-scripte/` liegt das Erzeugungsskript `erzeuge_assemblerdokumentation_pdf.py`
`Sourcecode/`	C-Quellcode des Assemblers selbst; enthält außerdem die Makrodatei `c82ass.makros` (der Assembler sucht sie unter diesem festen Namen zuerst im Arbeitsverzeichnis des Aufrufs, sonst in `Sourcecode/`; hier liegt sie richtig)
`_Linux/`, `_Windows/`	Binaries des Assemblers je Betriebssystem
`projekdetails`, `projekdetails.<Projekt>`	Arbeitsdateien: zuletzt benutzte Aufrufparameter, bzw. für ein bestimmtes Projekt gesicherte
`Projekt_1/`, `Projekt_2/`, `Proj_FeldSortieren/`, `Claude/`	Projektverzeichnisse (Beispiele); dürfen beliebig benannt sein und Unterverzeichnisse enthalten
`Claude_bsp_reassemblierung/`	Projektverzeichnisse, je eines pro Programm; enthält u. a. die Quelltexte der Tests aus `Tests/`

Vom Assembler erzeugte Dateien (im Verzeichnis der `.asm`-Datei, Namensstamm wie die Quelle; der Assembler löscht `.log`, `.obj`, `.mqo`, `.sym`, `.test` und `.dmp` vor jedem Lauf und erzeugt sie neu):

Endung	Inhalt
`.asm`	Quelltext (Eingabe, keine erzeugte Datei)
`.log`	Literaltabelle, Symboltabelle, Tabelle der absoluten Symbole
`.obj`	ladefähiger Objektcode (K1K2)
`.mqo`	aufbereiteter Quelltext, kombiniert mit dem Maschinencode (Hilfsdatei, Pass 3)
`.sym`	Symboldatei (ordnet Symbole zu Trommeladressen zu)
`.dmp`	formatierte Speicherbelegung (Listing mit Adresse, Maschinenwort, Label, Mnemonic, Operand, Kommentar); entsteht in Pass 3
`.test`	Trommeldatei im Format des Interpreters (`Adresse: Wert # Quelltextzeile`) mit Platzhalter-Definitionen für Testmarken und Tastenfolge sowie ggf. dem Leerbefehl 164, die für den jeweiligen Fall ergänzt werden müssen; Grundlage für Testaufträge, s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a

3a. Teilprojekt BIS

Das Bausteininterpretationssystem BIS ist das dritte Teilprojekt (Zweck und Inhalt: `C8205-Interpreter-Projektueberblick.md`, Abschnitt 3a). Es besteht derzeit **nur aus Dokumentation**: kein Quellcode, kein Binary, keine erzeugten Dateien. Die Dokumentation liegt im Ordner `Bausteininterpretiersystem BIS/markdown-dateien/` als `C8205-Interpreter-BIS.md` (maßgeblich für das BIS) und `C8205-Interpreter-Objektdatei_Listing.md` (Format der Objektdatei, die das KEP liest, und Weg vom Vorgabebild zum Testauftrag); das zugehörige PDF `C8205-BIS_A4.pdf` liegt im selben Ordner, ebenso der Unterordner `pdf-layout-scripte/` mit dem Erzeugungsskript `erzeuge_bis_pdf.py`. Der Interpreter und der Assembler benutzen das BIS nicht. Das Minimalprogramm der BIS-Tests (`pj2-bis_minimalprogramm`) liegt als Assemblerprojekt in `Assembler/Projekt_2/`, die Testaufträge in `Tests/` (Abschnitt 2, „BIS-Tests“). Werden später BIS-Bausteine abgelegt (Quelltexte, Trommeldateien, Lochband-Objektcode), ist dafür hier ein Verzeichnis nach dem Muster von `Assembler/` festzulegen und in diesem Abschnitt zu beschreiben.

4. Offene Punkte

Keine offenen Punkte.

Erledigt:

- **Bauskripte:** `c82ass` und `c82ass.bat` heißen jetzt `make_c82ass.sh` und `make_c82ass.bat` (Abschnitt 3). Die Assemblerdokumentation nennt die neuen Namen (.odt, Abschnitte 1.1 und 2); `.md` und `.pdf` sind daraus nachgeführt.
- **Assemblerdokumentation (.odt):** Änderungen (Passbeschreibung, Dateiliste, Streichung `.mqt`/`.l82`, `.dmp`, `.test`, Makrodatei, Beispiellauf, Aussage zu `c82ass.bat`) in die `.odt` übernommen. Das `.pdf` wurde daraus neu exportiert, die `.md` daraus nachgeführt.
- **Vorhandene `.mac`-Dateien:** vom Projektinhaber entfernt.
- **Ablageort der Makrodatei:** kein Problem. `erzeugeMacroArray` sucht `c82ass.makros` zuerst im Arbeitsverzeichnis und weicht auf `Sourcecode/` aus; die Ablage in `Assembler/Sourcecode/` ist damit richtig.
- `c82ass.sh`/`make_c82ass.bat` (die `.bat` hieß vormals `c82ass.bat`): geprüft; `.sh` startet nur das Binary, `.bat` erzeugt nur `c82ass.exe`.
- **Endungen `.dmp`, `.mac`, `.test`:** `.dmp` ist die formatierte Speicherbelegung (Pass 3), `.test` die Trommeldatei für den Interpreter (Abschnitt 3, Tabelle; ebenso in der Assemblerdokumentation), `.mac` wird nicht erzeugt.
- `.mqt`, `.l82`: vom aktuellen Assembler nicht mehr erzeugt, in der Assemblerdokumentation gestrichen.
- `c82load`, `c82dump`, `c82simi`, `C8205Simulator`: Diese Programme gehören nicht zum Projekt. Die Assemblerdokumentation (Abschnitt 1) erwähnt sie nur als Abnehmer von Assemblerdateien.

5. Nachführung Stufe 4 (19.10.2026)

Zeilen für `c8205_grafik.*`, `grafik_*.py`, `c-module/`, `ton/`, `pult/` und die GRAFIK-Prüfungen in `testscripte/` ergänzt (Abschnitt 1). Neues Beispielprogramm `Assembler/Projekt_1/sw_eingabe/k1_leser_stanzer.asm` (Lochband von Leser 1 auf Stanzer 1 kopieren, Ende mit : in KB) mit Test `Tests/k1_leser_stanzer.test`, Testband `virtuelle_Geraete/Leser1/lb_kopie_quelle.txt`, Soll `Test-Ergebnisvergleiche/k1_leser_stanzer-stanzer-ausgabe.txt`. Die PDFs erzeugt der Projektinhaber.

6. GRAFIK-Verzeichnisse (Stufe 6)

GRAFIK: Zum Modus GRAFIK gehören `grafik-interpreter-module/` (Module `grafik_*.py`), `c-module/` (echte Trommel, Tonmischer), `ton/` (Klänge und `sw_ton.py`), `pult/` (Grafiken und Manifest des Bedienpults) und `testscripte/` (GRAFIK-Prüfungen, Start über `grafik_alle_tests.sh`). Aufbau der Module: `C8205-Interpreter-GRAFIK_Beschreibung.md`, Abschnitt 11; Prüfungen: `C8205-Interpreter-GRAFIK_Pruefverfahren.md`.